

Arquitectura de Computadores · UNAL Medellín

Cache Aware vs Oblivious

Benchmark de Multiplicación Iterada de Matrices

Santiago Uribe Echavarría | Juan Pablo Robledo Meza

Arquitectura de Computadores • 2026

Objetivo: multiplicación iterada de matrices

$$B_{i+1} = A \cdot B_i, \quad i = 0, \dots, l - 1$$

con $A \in \mathbb{R}^{m \times m}$, $B_0 \in \mathbb{R}^{m \times n}$ e $l = \lceil 2m/n \rceil$ iteraciones.

Objetivo: comparar el rendimiento de siete kernels que implementan esta recurrencia con distintas estrategias de acceso a caché, en dos arquitecturas de hardware.

Dos enfoques para la jerarquía de caché

Cache Aware

El programador **conoce** los parámetros del hardware (tamaño de L1, L2, tamaño de línea) y diseña el algoritmo explícitamente para ellos.

- ▶ Bloques ajustados a tamaños de caché específicos
- ▶ Alto rendimiento en el hardware objetivo
- ▶ Requiere *re-tuning* al cambiar de máquina

Cache Oblivious

El algoritmo logra uso óptimo de caché **sin conocer** ningún parámetro de la jerarquía.

- ▶ Se adapta automáticamente a todos los niveles
- ▶ Un solo ajuste funciona en cualquier máquina
- ▶ Costo: overhead recursivo + pre-reorganización de datos

Layout row-major: buffer plano `scalar_t *`

M_{00}	M_{01}	M_{02}	M_{10}	M_{11}	$\dots$
----------	----------	----------	----------	----------	---------

$$M[i, j] \longrightarrow M[i * \text{cols} + j]$$

- ▶ `scalar_t = float` (4 bytes, IEEE 754)
- ▶ **Alineación de 64 bytes** via `posix_memalign`
⇒ `M[0]` siempre al inicio de una línea de caché
- ▶ 16 floats por línea de caché.

Layout alternativo: Morton

La familia *cache-oblivious* reorganiza **solo** *A* a Z-order antes del benchmark. *B* y *C* se mantienen row-major.

Esquema Anexo

Generador Lineal Congruencial (LCG)

$$s_{n+1} = a \cdot s_n + c \pmod{2^{32}}$$

$$a = 1\,664\,525 \quad c = 1\,013\,904\,223 \quad (\textit{Numerical Recipes})$$

- ▶ Completamente **determinístico**: la misma semilla produce la misma secuencia en cualquier plataforma
- ▶ Semilla 42 para A , semilla 43 para Z_0
- ▶ Garantiza que todos los kernels reciben *exactamente* las mismas matrices

¿Por qué no `rand()`?

`rand()` es dependiente de plataforma y no reproducible entre compiladores. Para un benchmark necesitamos entradas idénticas en local y en servidor.

Problema: medimos la recurrencia $B_{i+1} = A \cdot B_i$ con $l = 2m/n$ iteraciones. Sin control, $\|B_i\|$ crece exponencialmente $\rightarrow$ **overflow**.

Corolario 4.4.7 (Vershynin, 2018): Para $A \in \mathbb{R}^{m \times m}$ aleatoria con entradas subgaussianas de parámetro v :

$$\|A\|_2 \leq C v \sqrt{m} \quad \text{con alta probabilidad}$$

Solución: escalar las entradas por $1/\sqrt{m}$:

$$v = \frac{1}{\sqrt{m}} \Rightarrow \|A\|_2 = \Theta(1) \Rightarrow \|B_i\| \leq \|A\|^i \|B_0\| = O(1) \quad \forall i$$

- ▶ Sin escalado: $\|A\| = \Theta(\sqrt{m})$ y con $l = 2m/n$ iteraciones, $\|B_l\| \sim (\sqrt{m})^{2m/n}$ — overflow garantizado para m mediano
- ▶ Con escalado: la recurrencia es numéricamente estable sin importar m ni l

Parte I

Cache Aware

Visualización

Visualización interactiva del patrón de acceso a memoria para cada orden de loops ijk, ikj, jik, jki, kij, kji

Loop Reordering

Por qué ikj y el problema de escala

Mejor variante: ikj — los tres accesos del inner loop son favorables:

Variante	$C[i, j]$	$A[i, k]$	$B[k, j]$
ijk	escalar	stride 1	stride n
ikj	stride 1	escalar	stride 1
kji	stride m	stride m	escalar

¿Por qué `init_matrix_zero` es obligatorio en ikj ?

En ikj , la dimensión de reducción k *no* está en el loop interno. $C[i, j]$ se construye **acumulando** contribuciones parciales de k distintas. Sin el cero inicial, la primera acumulación sumaría sobre basura.

Visualización

Visualización del recorrido en bloques sobre A , B y C
con los paneles activos resaltados en cada paso

Acumulación por bloques:

$$C[i_i : i_i + M_c, :] += A[i_i : i_i + M_c, k_k : k_k + K_c] \cdot B[k_k : k_k + K_c, :]$$

con $M_c = K_c = 256 \Rightarrow$ los tres paneles activos llenan exactamente el **L2 (512 KB)**:

Panel	Dimensión	Tamaño
<i>A</i> activo	256×256	256 KB
<i>B</i> activo	256×128	128 KB
<i>C</i> activo	256×128	128 KB
Total		512 KB

Cada bloque aplica ijk internamente. Mejora visible a partir de $m \approx 512$.

¿Y si cada bloque lo multiplicáramos más rápido?

Visualización

Visualización del loop nest BLIS Goto-style
y del tile 6×16 recorriendo el bloque de C

Tiled AVX2 ¿Por qué 6×16 ?

Budget de registros YMM (Zen 2 tiene 16):

$$\underbrace{M_R \times 2}_{\text{acum. } C} + \underbrace{2}_B + \underbrace{1}_{A_{\text{bcast}}} \leq 16 \Rightarrow M_R \leq 6.5$$

$N_R = 16$: cada fila de B son 16 floats = 2 registros YMM.

Con $M_R = 6$, $N_R = 16$: **15 de 16 registros YMM usados**

Por qué `static inline`:

- Sin `inline`: el ABI fuerza *spill* de los 12 acumuladores YMM al stack por cada llamada (~ 240 ciclos)

Tile 6×16 en registros

c00..c51	12 YMM (acumuladores C)
b0, b1	2 YMM (fila de B)
a	1 YMM (broadcast A)
Total	15 / 16

Por cada paso p :

12 FMAs $\times$ 8 floats = 96 floats
actualizados en C sin tocar memoria

```
memset(C, 0, ...);                                // secuencial

#pragma omp parallel                               // fork UNICA vez
{
    for (kk = 0; kk < k; kk += KC) {               // todos los threads

        #pragma omp for schedule(static)           // filas entre threads
        for (ic = 0; ic < m; ic += MC)
            kernel_avx2_tiled_6x16(...);

        // <- barrera implicita al salir del omp for
    }
}                                                    // join
```

¿Por qué `kk` no se paraleliza?

Cada iteración de `kk` *acumula* sobre `C`. Si dos threads trabajaran en distintos `kk` simultáneamente $\Rightarrow$ write race sobre las mismas celdas de `C`.

El loop `kk` corre en lockstep; la barrera implícita al final del `omp for` garantiza que todos terminaron de acumular el tile `k` actual antes de avanzar.

Core físico vs. lógico (SMT)

6 cores físicos, 12 lógicos (SMT). Cada core tiene **2 FMAs** compartidas. Un kernel FMA-bound no escala con SMT.

Parte II

Cache Oblivious

Morton Base ¿Por qué es cache-oblivious?

Idea central: en lugar de fijar M_c, K_c al tamaño del L2, dividir A recursivamente hasta que el sub-bloque *naturalmente* quepa en caché — sin saber cuál.

La clave: A se reorganiza en **Z-order (Morton)**. Al dividir un bloque de lado $2h$ en sus 4 cuadrantes, cada cuadrante ocupa un segmento *contiguo* de h^2 elementos:

$$\text{offset}_{TL/TR/BL/BR} = \{0, 1, 2, 3\} \cdot h^2$$

La recursión navega A sumando offsets — **sin strides**.

`morton_encode( $i, j$ ):`
Intercala los bits de i y j :

$$\text{code}(i, j) = i_{p-1}j_{p-1} \cdots i_0j_0$$

(i, j)	código	cuadrante
$(0, 0)$	0	TL
$(0, 1)$	1	TR
$(1, 0)$	2	BL
$(1, 1)$	3	BR

B y C permanecen **row-major**.

Caso N (*split de la dimensión libre n*)

Si $n_{\text{bloque}} > a_{\text{dim}}$: dividir n a la mitad. A es **compartida**; las dos sub-columnas de C son disjuntas.

Caso MK (*split simultáneo de m y k en cuatro cuadrantes*)

$$C_{\text{top}} = A_{TL}B_{\text{top}} + A_{TR}B_{\text{bot}}, \quad C_{\text{bot}} = A_{BL}B_{\text{top}} + A_{BR}B_{\text{bot}}$$

Los offsets Morton de los cuadrantes son $\{0, 1, 2, 3\} \cdot h^2$ — **contiguos en memoria**.
Primer producto: *overwrite*. Segundo: *accumulate*.

Caso LEAF

$m \cdot k \cdot n \leq \text{threshold}$: kernel base ijk escalar con indexing Morton.

$$g_recursion_threshold = 32 \times 32 \times 128 = 131\,072$$

A esa profundidad el panel de A activo es ≈ 4 KiB $\rightarrow$ cabe holgado en L1d (32 KiB).

Configurable en runtime:

`matmul_morton_set_threshold(t)`

- ▶ **Muy bajo:** recursión excesiva, overhead de llamadas domina
- ▶ **Muy alto:** el leaf ve sub-bloques que no caben en L1 $\rightarrow$ misses

Nota

`set_threshold(0)` se ignora con warning: un threshold cero colapsaría toda la matriz al leaf, eliminando cualquier ventaja de localidad.

Visualización

Árbol de recursión del split Z-order
y cómo los cuadrantes de A se visitan en orden de Morton

Cota óptima de transferencias de memoria para multiplicar matrices $m \times m$:

$$Q(m, M, B) \geq \Omega\left(\frac{m^3}{\sqrt{M} \cdot B}\right)$$

donde M = tamaño del caché, B = tamaño de bloque de transferencia.

Morton alcanza esa cota

(demostrado en Frigo et al., 1999):

La recursión Z-order realiza exactamente $\Theta\left(m^3 / \sqrt{M} \cdot B\right)$ transferencias **para cualquier nivel de la jerarquía simultáneamente**.

No hay ningún algoritmo que use menos I/Os.

Morton es, en ese sentido, **óptimo sin parámetros**.

Contraste con Tiled

`tilted_ikj` alcanza la misma cota, pero *solo para el nivel de caché* al que están ajustados M_c y K_c .

Morton la alcanza para L1, L2, L3 y RAM a la vez.

¿Por qué $M_R = 4$ y no 6 como en tiled?

1. **Compatibilidad con la recursión:** los splits dividen m por 2 repetidamente. Con m potencia de 2 y $M_R = 4$ (también potencia de 2), los sub-bloques siempre son múltiplos de M_R . Con $M_R = 6$ esto falla.
2. **Pre-fetch de A :** con $M_R = 4$ caben **4 broadcasts simultáneos** de A en registros, reduciendo dependencias RAW.

Layout Morton-de-bloques (tile 4×4):

$$A[i, j] \rightarrow A_M[\text{enc}(i/4, j/4) \cdot 16 + (i \% 4) \cdot 4 + (j \% 4)]$$

Balance de registros YMM

c00..c31	8 YMM (acum. C)
b0, b1	2 YMM (fila de B)
a0..a3	4 YMM (broadcast A)
Total	14 / 16

Threshold

Default: $64 \times 64 \times 128 = 524\,288$
Panel A activo ≈ 16 KiB $\rightarrow$ L1d. (32 KiB)

OMP tasks, no omp for:

La recursión Morton no tiene un loop rectangular. Los 4 sub-productos del caso MK tienen dependencias:

- ▶ TL → TR (acumula sobre TL)
- ▶ BL → BR (acumula sobre BL)
- ▶ **Top y Bottom son disjuntos** → sí pueden ir en paralelo

Se lanzan **2 tasks**: `mk_top_pair` y `mk_bot_pair`.

Dos thresholds independientes:

- ▶ `g_recursion_threshold_omp`: tamaño del leaf

Scratch pool per-thread

Cada hoja necesita un buffer `A_local` para materializar el panel Morton-de-bloques. En multi-hilo, dos hojas concurrentes no pueden compartirlo.

Solución: array de `max_threads` buffers; cada hilo indexa el suyo con `omp_get_thread_num()`.

Parte III

Experimentación

Local — Ryzen 5 4600H (Zen 2)

- ▶ 6 cores físicos / 12 hilos (**SMT**)
- ▶ L1d: 32 KB L2: 512 KB L3: 8 MB (2 CCX)
- ▶ **AVX2** — registros YMM (256 bits)
- ▶ 8 floats/registro, 2 FMA/core
- ▶ $M_R = 6$, $N_R = 16$, $K_c \approx 384$

Servidor — EPYC 9R45 (Zen 5, KVM)

- ▶ 8 cores / 8 hilos (**sin SMT**)
- ▶ L1d: 48 KB L2: 1 MB L3: 32 MB (shared)
- ▶ **AVX-512** — registros ZMM (512 bits)
- ▶ 16 floats/registro, 2 FMA/core (ancho doble)
- ▶ $M_R = 6$, $N_R = 32$, K_c mayor

Aspecto	main (local)	main_server
Hardware	Ryzen 5 4600H / Zen 2	EPYC 9R45 / Zen 5 (AWS)
ISA SIMD	AVX2 + FMA	AVX-512 + FMA
Microkernels	4×16 (Morton), 6×16 (tiled)	4×32 (Morton), 6×32 (tiled)
Default OMP	6 threads	8 threads
Setup	WSL2 + Ubuntu	Amazon Linux 2023 nativo

Gráficas

Métricas en Local y Server

Arquitectura de Computadores · UNAL Medellín

Gracias

Santiago Uribe Echavarría | Juan Pablo Robledo Meza